

LAPORAN AKHIR
TUGAS KECIL 1 : PENYELESAIAN PERMAINAN *QUEENS*
LINKEDIN
IF2211 STRATEGI ALGORITMA



Dosen Pembimbing:

Ir. Rila Mandala, M.Eng., Ph.D.

Disusun oleh:

Josh Reinhart Zidik 13524048

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

Pendahuluan

Tugas kecil 1 IF2211 Strategi Algoritma pada laporan ini adalah mengenai pengaplikasian algoritma *Brute Force* pada permainan *Queens* LinkedIn. Laporan ini ditulis untuk menjelaskan alur algoritma yang telah dibuat, *source code* dari program, serta hasil dari *test case* di dalam program.

Aturan Permainan

Queens LinkedIn adalah sebuah permainan logika yang terdapat di dalam aplikasi atau situs LinkedIn. Di dalam permainan Queens diberikan sebuah papan berukuran $n \times n$ dengan daerah atau warna sebanyak n . Tujuan dari permainan ini adalah untuk menempatkan n buah bidak ratu di dalam papan sehingga tidak ada satupun bidak yang memiliki baris, kolom, dan warna yang sama dengan bidak lainnya. Selain itu, bidak ratu juga tidak boleh bersebelahan secara diagonal dengan bidak lainnya.

Algoritma Penyelesaian Permainan

Algoritma yang ditulis pada tugas kecil ini menggunakan bahasa pemrograman Java untuk pemrosesan *backend* dan juga library JavaFX untuk pemrosesan GUI. Untuk memudahkan penyelesaian, dua kelas khusus dibentuk yaitu kelas Tile dan kelas Region. Potongan kode di bawah menunjukkan struktur dari setiap kelas di dalam program.

```
1 public class Tile {
2     public int row, col;
3     public char region;
4     public Tile(int row, int col, char region){
5         this.row = row;
6         this.col = col;
7         this.region = region;
8     }
9 }
```

```
1 public class Region {
2     public char name;
3     public Tile[] loc;
4     public int count;
5     public Region(char name, Tile[] loc, int count){
6         this.name = name;
7         this.loc = loc;
8         this.count = count;
9     }
```

Kelas *Tile* berfungsi untuk menyimpan informasi dari suatu kotak di dalam papan, sedangkan kelas *Region* berfungsi untuk menyimpan *array* dari kelas *Tile* beserta dengan informasi namanya.

Pada program yang telah ditulis, terdapat dua algoritma *brute force* dengan masing-masing algoritma menggunakan metode yang berbeda. Algoritma yang pertama adalah algoritma *brute force* murni yang menempatkan n buah bidak ratu dalam papan, sedangkan algoritma yang kedua adalah algoritma *brute force* yang berdasarkan warna di dalam papan. Kedua algoritma *brute force* menggunakan pendekatan yang sama yaitu iterasi pohon kemungkinan lokasi ratu menggunakan rekursi berulang sedalam n .

Metode Brute Force Murni

Metode ini mencoba setiap kombinasi penempatan n buah bidak ratu ke dalam papan berukuran $n \times n$ kemudian mengecek apabila kombinasi n bidak ratu yang ditempatkan memenuhi aturan atau tidak. Secara matematis, metode ini merupakan metode dengan kompleksitas waktu paling tinggi karena terdapat $C(N^2, N)$ kombinasi yang mungkin. Oleh karena itu, metode ini dapat berkembang dengan sangat cepat dan tidak efisien untuk papan berukuran besar.

Metode ini ditulis melakukan iterasi “pohon kemungkinan” menggunakan rekursi. Pohon kemungkinan tersebut memiliki akar yaitu bidak ratu ke-0 atau yang pertama dan ratu ke- i untuk tingkat pohon ke- i . Agar tidak ada kombinasi yang diperiksa dua kali, maka setiap bidak ratu diurutkan di dalam papan, sehingga ratu ke- j tidak akan muncul sebelum ratu ke- i jika $i < j$. Tabel di bawah merupakan visualisasi posisi bidak ratu seiring berjalannya algoritma *brute force* pada papan berukuran 4×4 .

Kombinasi 1			
#	#	#	#

Kombinasi 2			
#	#	#	
#			

Kombinasi 3			
#	#	#	
	#		

--	--	--	--

--	--	--	--

--	--	--	--

Kombinasi 4			
#	#	#	
		#	

Kombinasi 13			
#	#	#	
			#

Kombinasi 14			
#	#	#	
#	#		

Kombinasi 24			
#	#	#	
#			
			#

Kombinasi 25			
#	#	#	
	#	#	

Kombinasi 14			
#	#	#	
		#	#

Pada visualisasi di atas, bidak ratu ke-4 berjalan sampai ke ujung papan sebelum bidak ratu ke-3 berjalan satu petak. Hal yang sama berlaku untuk bidak ratu ke-2, agar dapat berjalan satu petak maka bidak ratu ke-3 harus berjalan sampai ke ujung papan. Oleh karena ukurannya yang bersifat faktorial, metode ini dapat berkembang menjadi sangat besar ketika jumlah ratu ditambahkan ke dalam papan permainan. Gambar berikut adalah potongan kode dari metode *brute force* murni untuk penyelesaian permainan *queens*.

```

1 public static Tile[] bruteMethod(int depth, char[][] board,
2                                 Tile[] current, int prev,
3                                 boolean isGUI, SolverController controller) throws IOException{

```

Gambar di atas merupakan *header* dari fungsi yang digunakan untuk penyelesaian permainan menggunakan metode *brute force* murni. Argumen **depth** berfungsi untuk menyimpan kedalaman rekursi yang sedang berjalan serta menunjukkan bidak ratu mana yang sedang diproses, program akan terus melakukan rekursi hingga parameter tersebut sama dengan *n* atau jumlah bidak ratu maksimum pada papan tersebut. Argumen **board** berfungsi sebagai representasi dari papan permainan, setiap warna dijadikan sebuah data char. Selanjutnya, argumen **current** berfungsi untuk menyimpan data lokasi bidak ratu sebagai *array* di dalam rekursi, sementara argumen **prev** berfungsi untuk menyimpan data lokasi ratu sebelumnya pada rekursi, argumen ini dibutuhkan agar program tidak mengecek kombinasi ratu yang

sama dua kali atau lebih. Terakhir, argumen *isGUI* dan *controller* tidak berfungsi untuk algoritma *brute force*, sebaliknya digunakan untuk membantu *live update* pada GUI.

```
1  int n = board.length;
2  if(depth == n){
3      steps++;
4      if(steps % (totalstep / 100) == 0){
5          io_file.updateBoard(isGUI, current, board, steps, totalstep, controller);
6      }
7      if(isSolutionValid(current) && isSolutionValid2(current)) return current;
8      else return null;
9  }
```

Gambar selanjutnya menunjukkan algoritma basis pada rekursi yang hanya terpenuhi jika argumen *depth* sama dengan jumlah bidak ratu maksimum, yaitu jumlah baris dan kolom pada papan. Ketika rekursi sudah mencapai basis, kombinasi lokasi ratu di dalam argumen *current* diperiksa. Jika kombinasi tersebut memenuhi semua aturan maka fungsi akan mengembalikan argumen *current*, namun jika kombinasi salah maka fungsi akan mengembalikan null.

```
1  else{
2      Tile[] result = new Tile[n];
3      for(int i = prev + 1; i <= n * (n-1) + depth; i++){
4          int row = i / n;
5          int col = i % n;
6          Tile queen = new Tile(row, col, board[row][col]);
7          current[depth] = queen;
8          result = bruteMethod(depth + 1, board, current, i, isGUI, controller);
9          if(result != null) return result;
10     }
11     return result;
12 }
```

Potongan kode di atas menunjukkan algoritma yang dijalankan oleh program jika belum mencapai basis. Di dalam algoritma tersebut dijalankan perulangan dari *prev + 1* (posisi setelah ratu sebelumnya) sampai $n * (n-1) + \text{depth}$ (posisi ratu terakhir yang mungkin). Setiap iterasi dari perulangan tersebut akan memanggil fungsi sendiri dengan argumen yang sama kecuali *depth + 1* untuk menunjukkan bidak ratu selanjutnya yang akan ditempatkan. Selain itu, posisi ratu ke-*i* dimasukkan ke dalam argumen *current* sebelum fungsi selanjutnya dipanggil. Terakhir, hasil dari rekursi diperiksa

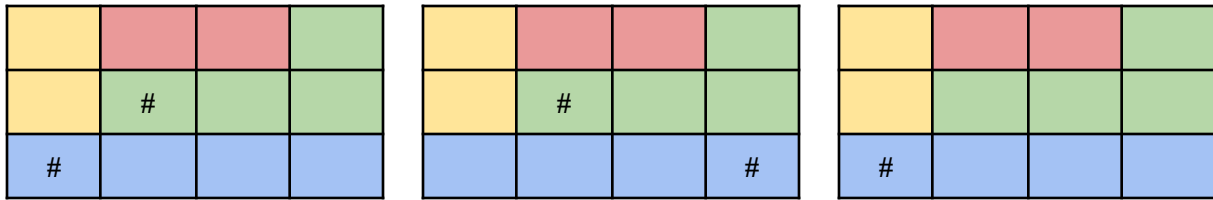
sebelum dikembalikan, jika hasil yang diterima bukan *null* artinya solusi valid telah ditemukan sehingga perulangan akan berhenti berjalan dan fungsi akan langsung keluar menghasilkan solusi yang valid.

Metode *Brute Force* Berdasarkan Warna

Metode *brute force* ini menggunakan aturan bahwa setiap bidak ratu dipastikan terdapat tepat 1 pada setiap daerah warna, n jumlah ratu ditempatkan di papan dengan masing-masing bidak berada di warnanya masing-masing. Metode ini dapat dianggap sebagai *brute force* yang tidak murni karena mengandung optimisasi dan aturan tambahan, akan tetapi memiliki kompleksitas algoritma yang lebih kecil. Kompleksitas algoritma dari metode *brute force* ini adalah $O(N^N)$ karena pada kondisi terburuk setiap warna memiliki tepat n buah kotak.

Perbedaan utama antara metode *brute force* ini dengan metode *brute force* murni adalah pada jumlah perulangan yang dijalankan setiap rekursi. Metode berdasarkan warna melakukan perulangan sebanyak jumlah kotak untuk warna ke- i sementara metode murni melakukan perulangan sebanyak jumlah kotak pada papan. Tabel di bawah adalah visualisasi bagaimana metode *brute force* berdasarkan warna bekerja seiring dengan berjalannya program.

Kombinasi 1				Kombinasi 2				Kombinasi 3			
#	#		#	#	#		#	#	#		#
#					#					#	
Kombinasi 4				Kombinasi 5				Kombinasi 8			
#	#		#	#	#			#	#		
							#				#
				#							#
Kombinasi 9				Kombinasi 12				Kombinasi 13			
#	#			#	#			#		#	#



Metode ini secara umum lebih cepat dalam menemukan solusi karena sudah memastikan bahwa satu aturan sudah terpenuhi di dalam setiap kombinasinya. Pada kondisi terburuk, setiap warna dari papan memiliki jumlah yang sama, yaitu n sehingga menghasilkan n^n kombinasi bidak ratu yang mungkin (sehingga kompleksitas algoritmanya adalah $O(N^N)$). Gambar berikut adalah potongan kode dari metode penyelesaian permainan berdasarkan warna papan.

```

1 public static Tile[] regionMethod(int depth, Region[] regions,
2                                   Tile[] current, char[][] board,
3                                   boolean isGUI, SolverController controller) throws IOException {

```

Potongan kode di atas menunjukkan *header* dari metode *brute force* berdasarkan warna. Serupa dengan metode murni, argumen *depth* digunakan untuk mengukur kedalaman rekursi dan argumen *current* digunakan untuk menyimpan lokasi bidak ratu dalam tiap rekursi. Selanjutnya, argumen *regions* serta *board* digunakan sebagai informasi utama papan dan kelompok (*array*) warna setiap petak di dalamnya.

```

1 boolean found = false;
2 Tile[] result = new Tile[regions.length];
3 if(depth == regions.length){
4     steps++;
5     if(steps % (totalstep / 100) == 0){
6         io_file.updateBoard(isGUI, current, board, steps, totalstep, controller);
7     }
8     if(isSolutionValid(current) && isSolutionValid2(current)) return current;
9     else return null;
10 }

```

Selanjutnya, potongan kode di atas merupakan algoritma *brute force* ketika sudah mencapai basis rekursi. Basis rekursi tercapai jika tepat n buah ratu telah terpilih ($depth = n$) sehingga kombinasi dapat divalidasi.

```
1  else{
2      int i = 0;
3      while(i < regions[depth].count && !found){
4          current[depth] = regions[depth].loc[i];
5          result = regionMethod(depth + 1, regions, current, board, isGUI, controller);
6          if(result != null) found = true;
7          i++;
8      }
9      return result;
10 }
```

Algoritma rekursif pada potongan kode di atas adalah algoritma yang membedakan metode *brute force* murni dan metode *brute force* berdasarkan warna. Pada metode ini, perulangan dilakukan tidak kepada seluruh papan melainkan kepada petak di daerah ke-*depth*.

Algoritma Lainnya

Selain algoritma kedua metode *brute force*, program ini juga memiliki sejumlah algoritma lainnya yang berkaitan untuk membantu berjalannya algoritma *brute force*.

Algoritma Validasi Ratu

Algoritma ini berfungsi untuk memvalidasi jika kombinasi bidak ratu pada papan sudah sesuai dengan aturan atau belum. Validasi ini mengikuti aturan dari permainan *queens* yaitu setiap baris, kolom, warna dalam papan harus memiliki **tepat satu** jumlah bidak ratu dan setiap ratu tidak boleh bersebelahan termasuk secara diagonal. Gambar di bawah merupakan potongan kode untuk mengecek jika setiap ratu berada di baris, kolom, dan warna yang berbeda.

```
1  public static boolean isSolutionValid(Tile[] queens){
2      int regioncount = 0;
3      boolean[] row = new boolean[queens.length];
4      boolean[] col = new boolean[queens.length];
5      char[] region = new char[queens.length];
6      for(int i = 0; i < queens.length; i++){
7          int j = 0;
8          if(row[queens[i].row]) return false;
9          if(col[queens[i].col]) return false;
10         while(j < regioncount){
11             if(queens[i].region == region[j]) return false;
12             j++;
13         }
14         region[j] = queens[i].region;
15         regioncount++;
16         row[queens[i].row] = true;
17         col[queens[i].col] = true;
18     }
19     return true;
20 }
```

Validasi ini bekerja dengan menggunakan *array of boolean* untuk pengecekan baris dan kolom serta *array of char* untuk pengecekan warna. Saat deklarasi, setiap indeks pada *array* baris dan kolom memiliki nilai *false* sehingga jika terdapat bidak ratu yang memiliki baris ke-*i* dan kolom ke-*j*, maka indeks ke-*i* pada *array* baris dan indeks ke-*j* pada *array* kolom akan dijadikan *true*. Jika terdapat bidak ratu yang memiliki baris atau kolom yang sama, akan mendapatkan hasil *true* pada indeks *array* baris atau kolom, sehingga akan mengembalikan nilai *false* dan keluar dari program. Berbeda dengan validasi baris dan kolom, validasi warna menggunakan *array of char* yang menyimpan warna dari bidak ratu yang sudah diperiksa. Jika bidak ratu yang diperiksa ternyata memiliki warna yang sudah disimpan, maka fungsi akan mengembalikan *false* dan keluar. Gambar selanjutnya adalah algoritma validasi jika ratu

bersebelahan secara diagonal, kondisi ratu bersebelahan secara vertikal ataupun horizontal tidak perlu diperiksa karena sudah dipenuhi oleh kondisi sebelumnya.

```
1 public static boolean isSolutionValid2(Tile[] queens){
2     boolean[][] illegal = new boolean[queens.length][queens.length];
3     int i = 0;
4     while(i < queens.length){
5         int row = queens[i].row;
6         int col = queens[i].col;
7         if(illegal[row][col]) return false;
8         if(col > 0 && row > 0) illegal[row-1][col-1] = true;
9         if(col < queens.length-1 && row > 0) illegal[row-1][col+1] = true;
10        if(col > 0 && row < queens.length-1) illegal[row+1][col-1] = true;
11        if(col < queens.length-1 && row < queens.length-1) illegal[row+1][col+1] = true;
12        i++;
13    }
14    return true;
15 }
```

Validasi ini menggunakan *array of array of boolean* yang menggambarkan kondisi papan ketika bidak ratu ditempatkan. Saat salah satu bidak ratu ditempatkan, maka petak valid (tidak keluar papan) yang bersebelahan secara diagonal akan ditandai sebagai *illegal*. Jika terdapat ratu selanjutnya yang ditempatkan pada petak *illegal* maka fungsi akan mengembalikan *false* dan langsung keluar.

Algoritma Validasi Papan

Algoritma validasi papan juga dibutuhkan agar dapat dipastikan bahwa algoritma penyelesaian permainan dapat berjalan tanpa menemui *error* yang tidak diinginkan. Karena aturan permainan ini mengharuskan terdapat tepat satu bidak ratu di setiap baris, kolom, dan warna, maka papan dipastikan harus memiliki jumlah baris, kolom, dan warna yang sama. Gambar di bawah merupakan algoritma pengecekan papan yang terutama, di dalamnya terdapat dua algoritma berbeda untuk validasi.

```
1 public static boolean isBoardValid(char[][] board){
2     if(!isBoardSizeValid(board)){
3         System.out.println("Ukuran papan tidak valid!");
4         return false;
5     }
6     try {
7         Region[] regions = Region.getRegions(board);
8         return true;
9     } catch (Exception e){
10        System.out.println(e.getMessage());
11        return false;
12    }
13 }
```

Gambar berikutnya merupakan algoritma dari **isBoardSizeValid()** yang berfungsi untuk memastikan bahwa input benar merupakan papan yang berbentuk persegi. Algoritma dari **Region, getRegions()** akan dijelaskan pada subbab berikutnya.

```
1 public static boolean isBoardSizeValid(char[][] board){
2     int i = 1;
3     int Nlength = board[0].length;
4     while(i < board.length && board[i] != null){
5         if(board[i].length != Nlength) return false;
6         i++;
7     }
8     return (i == Nlength);
9 }
```

Algoritma di atas memastikan bahwa setiap baris pada papan memiliki jumlah petak yang sama, jika terdapat baris dengan jumlah petak yang berlebih atau kurang fungsi akan langsung mengembalikan *false*. Selain itu, fungsi memastikan bahwa jumlah baris sama dengan jumlah kolom sehingga tidak terdapat papan yang berbentuk persegi panjang.

Algoritma Pembagian Warna

Algoritma ini berfungsi untuk membagi petak-petak pada papan menjadi *region* atau daerah yang ditandai dengan kelas *Region* di dalam algoritma. Algoritma mula-mula akan melakukan iterasi untuk setiap petak di dalam papan, kemudian pada setiap iterasi tersebut akan memanggil fungsi **updateRegion()** untuk menambahkan petak tersebut ke dalam sebuah kelas *Region*.

```
public static Region[] getRegions(char[][] board){
    int i = 0;
    Region[] regions = new Region[board.length];
    System.out.println();
    while(i < board.length){
        int j = 0;
        while(j < board[i].length){
            Tile loc = new Tile(i, j, board[i][j]);
            updateRegion(loc, board, regions);
            j++;
        }
        i++;
    }
    int regioncount = 0;
    while(regioncount < regions.length){
```

```

        if(regions[regioncount] == null) throw new
IllegalArgumentException("Jumlah region kurang");
        regioncount++;
    }
    for(int k = 0; k < regions.length; k++){
        regions[k].loc = Arrays.copyOf(regions[k].loc,
regions[k].count);
    }
    return regions;
}
}

```

Potongan kode di bawah adalah algoritma dari **updateRegion()**

```

    public static void updateRegion(Tile loc, char[][] board, Region[]
regions){
        int i = 0;
        int j = 0;
        char name = board[loc.row][loc.col];
        updateRegionList(name, regions);
        while(i < regions.length && regions[i].name != name) i++;
        if (i == regions.length || regions[i] == null) throw new
IllegalArgumentException("Region terlalu banyak"); // region penuh, gak
bisa update
        while(regions[i].loc[j] != null) j++;
        if (j < regions[i].loc.length){
            regions[i].loc[j] = loc;
            regions[i].count++;
        }
    }
}

```

```

    public static boolean updateRegionList(char name, Region[] regions){
        // cek apakah region udah ada, kalau belum langsung ditambahin,
false kalau region sudah ada, true kalau region belum ada
        int i = 0;
        while(i < regions.length && regions[i] != null){
            if(regions[i].name == name) return false;
            i++;
        }
        if (i < regions.length) regions[i] = new Region(name, new
Tile[1024], 0);
    }
}

```

```
    return true;  
}
```

Hasil Test Case

Test Case 1 (Papan invalid)

Untuk *test case* 1 GUI hanya menampilkan pop-up untuk menandai bahwa input invalid dan meminta untuk menginput ulang.



	Input	Output	Keterangan
1	AAAAAA BBBBB CCCCCCC DDDDDD EEEEEEEE	Masukkan nama file: invalid_board_1.txt Ukuran papan tidak valid! Papan invalid! Keluar dari program...	Input tidak memiliki jumlah kolom konsisten
2	AAAAA BBBBB CCCCC DDDDD EEEEEE FFFFFF GGGGG	Masukkan nama file: invalid_board_2.txt Ukuran papan tidak valid! Papan invalid! Keluar dari program...	Input tidak berbentuk persegi
3	AAAAA BBBBC DDDC DCCCE FFFE	Masukkan nama file: invalid_board_3.txt Region terlalu banyak Papan invalid! Keluar dari program...	Jumlah warna melebihi jumlah baris atau kolom
4	AAAAA AAAAC BBBBB DDDDD BBBBB	Masukkan nama file: invalid_board_4.txt Jumlah region kurang Papan invalid! Keluar dari program...	Jumlah warna kurang dari jumlah baris atau kolom

Test Case 2 (Papan 5 x 5 valid)

INPUT:

board_5.txt

AABBB

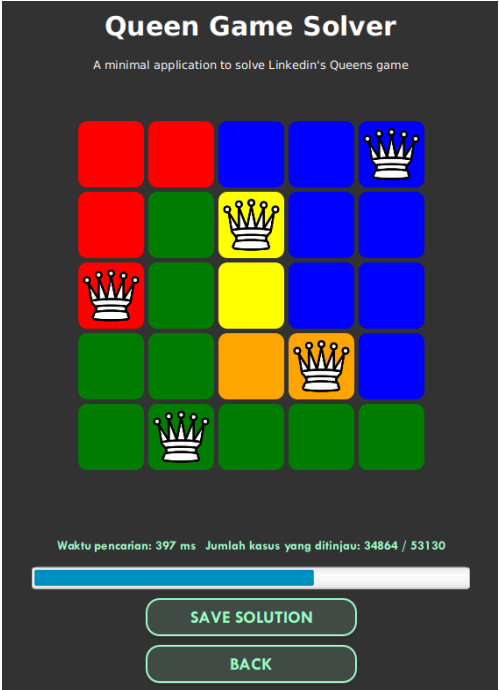
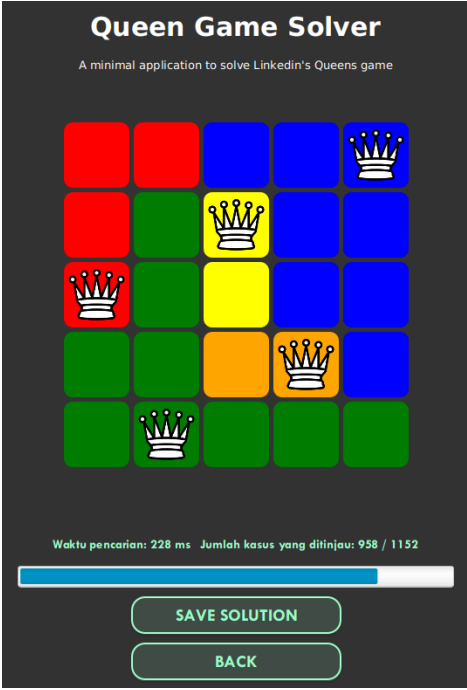
ACDBB

ACDBB

CCEEB

CCCCC

OUTPUT:

<i>Brute Force</i> Murni	<i>Brute Force</i> Berdasarkan Warna
	

Test Case 3 (Papan 6 x 6 valid)

INPUT:

board_6.txt

ABBBAA

AAAAAA

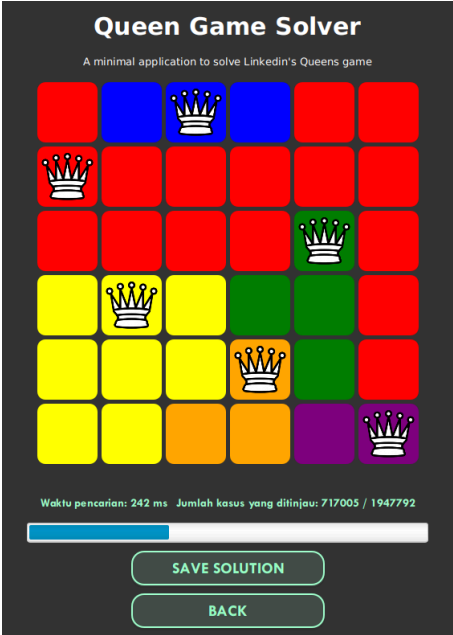
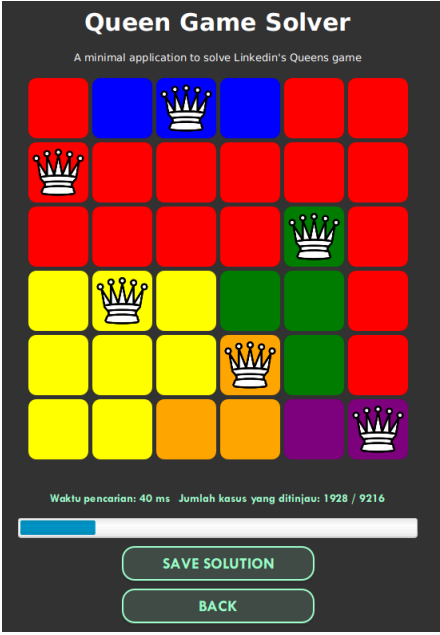
AAAACA

DDCCCA

DDDECA

DDEEFF

OUTPUT:

Brute Force Murni	Brute Force Berdasarkan Warna
	

Test Case 4 (Papan 7 x 7 valid)

INPUT:

board_7.txt

AAABCDD

ABBBBCDD

AEEEECCD

AEECCDD

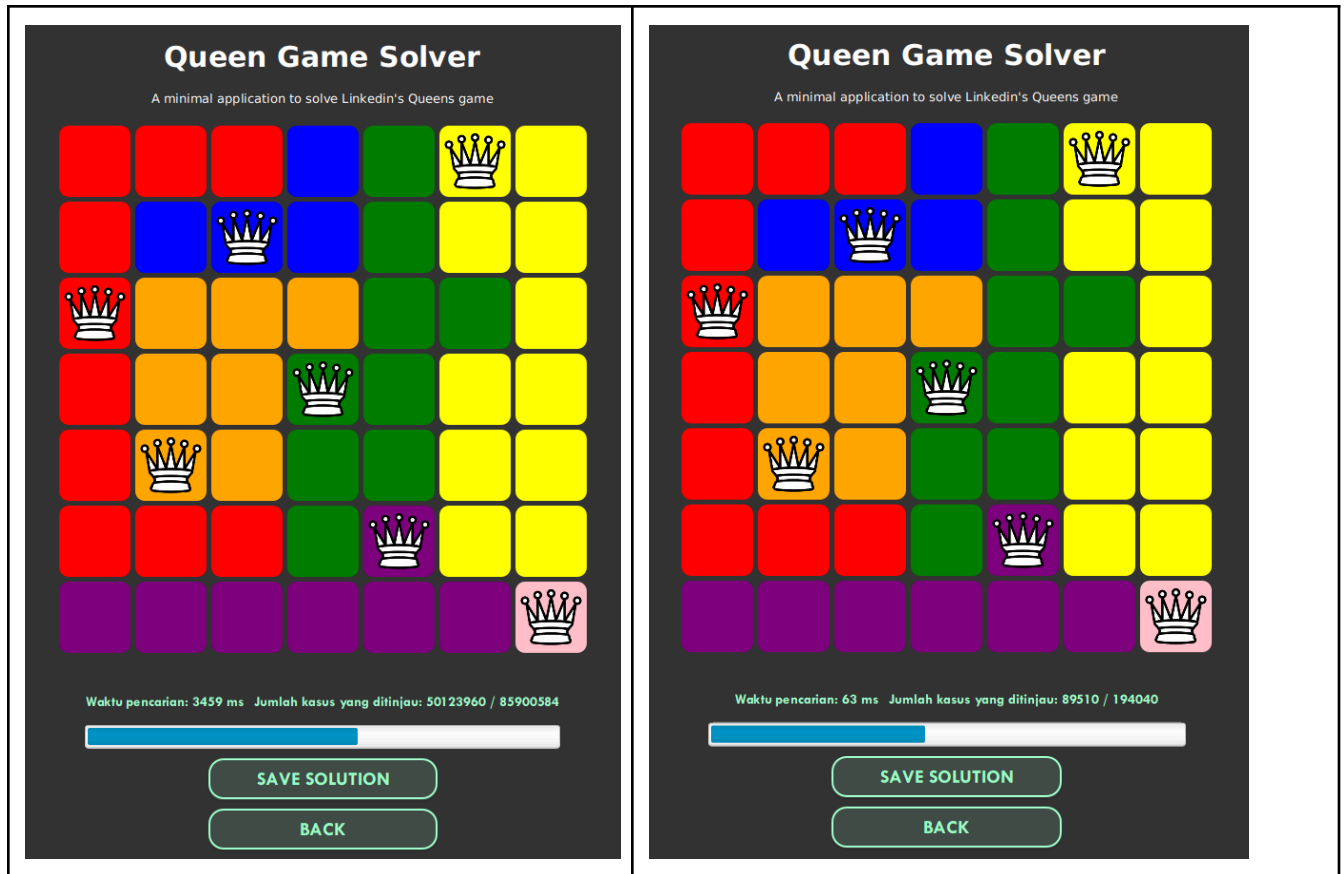
AEECCDD

AAACFDD

FFFFFFFG

OUTPUT:

Brute Force Murni	Brute Force Berdasarkan Warna
-------------------	-------------------------------



Test Case 5 (Papan 8 x 8 valid)

INPUT:

board_8.txt

AABBBCCC

ADBDBECC

ADBDBCCC

ADDDBFGC

ADDDBFGG

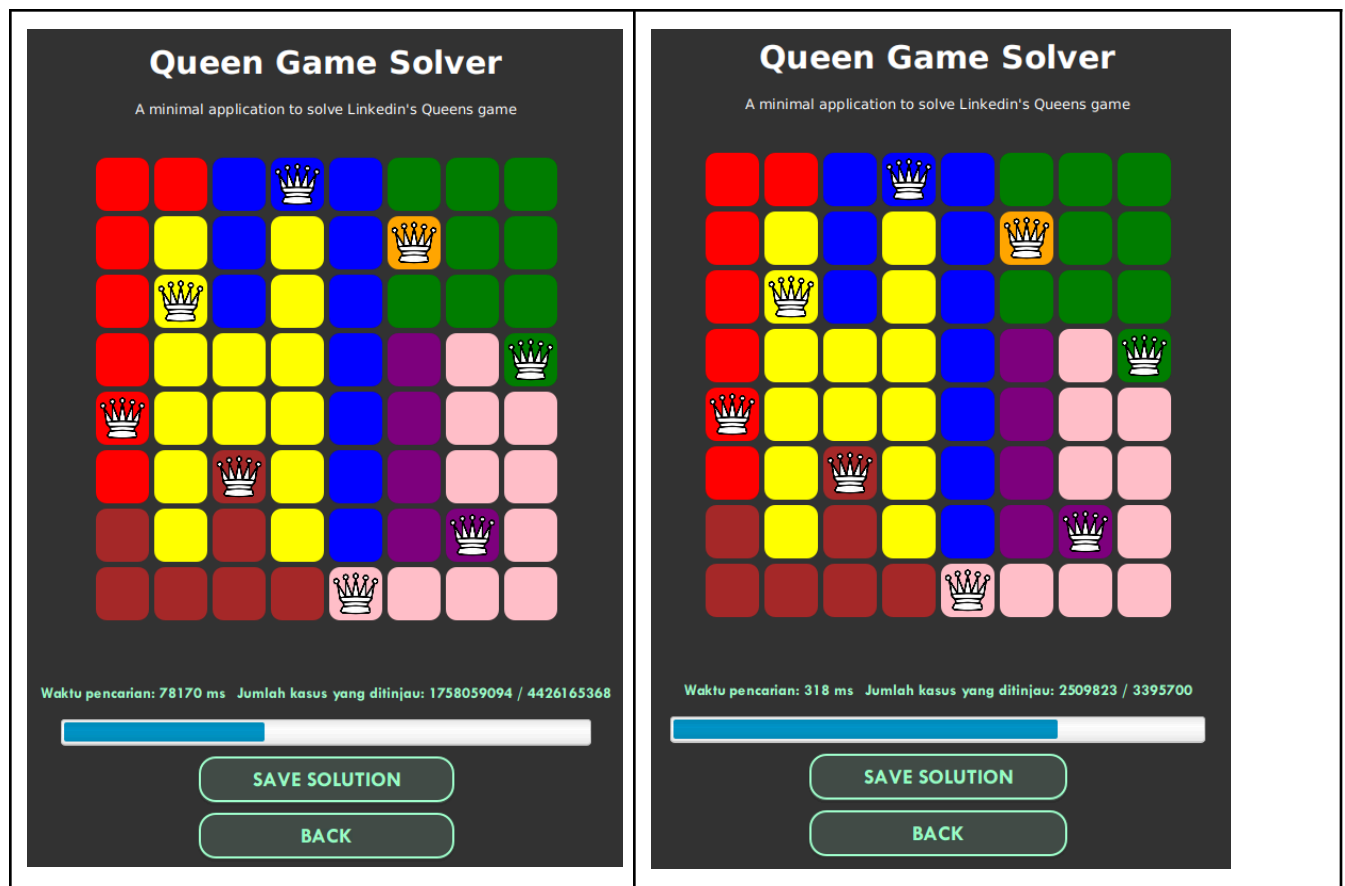
ADHDBFGG

HDHDBFFG

HHHHGGGG

OUTPUT:

<i>Brute Force</i> Murni	<i>Brute Force</i> Berdasarkan Warna
--------------------------	--------------------------------------



Test Case 6 (Papan 6 x 6 tanpa solusi)

INPUT:

no_solution_1.txt

ABCCCC

CCCCCC

DDDDDD

EEEEEE

FFFFFF

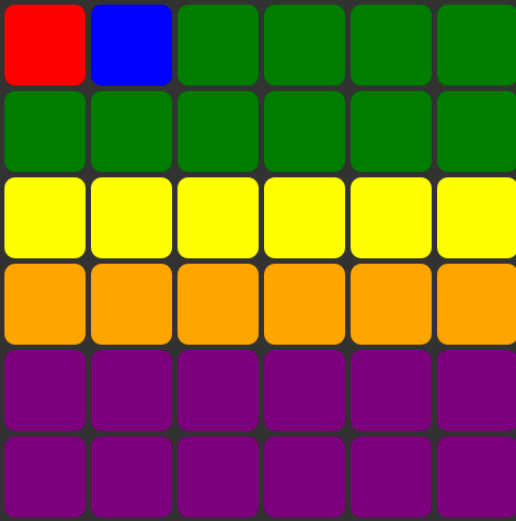
FFFFFF

OUTPUT:

<i>Brute Force Murni</i>	<i>Brute Force Berdasarkan Warna</i>
---------------------------------	---

Queen Game Solver

A minimal application to solve LinkedIn's Queens game



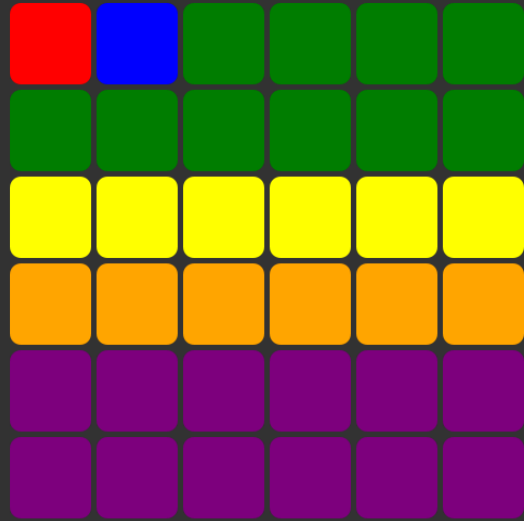
Waktu pencarian: 500 ms Jumlah kasus yang ditinjau: 1947792 / 1947792

SOLUTION NOT FOUND!

BACK

Queen Game Solver

A minimal application to solve LinkedIn's Queens game



Waktu pencarian: 100 ms Jumlah kasus yang ditinjau: 4320 / 4320

SOLUTION NOT FOUND!

BACK

Lampiran

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Josh Reinhart Zidik

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil di jalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Program dapat menyimpan solusi dalam bentuk file gambar		✓

Pranala repository program: <https://github.com/Achideon/IF2211-Tucil-1-Stima>